

计算机网络

实验一：数据链路层滑动窗口协议的设计与实现

小组成员 张恒基、尹浩铭、林旭东

班 级 （填写班级）

日 期 2026 年 5 月

目 录

1	11.1 实验内容和实验环境描述	3
	1.1 实验任务与目标	3
	1.2 实验环境	3
2	11.2 软件设计	4
	2.1 (1) 数据结构	4
	2.2 (2) 模块结构	4
	2.3 (3) 算法流程与异常路径	5
3	11.3 实验结果分析	6
	3.1 有误码下的正确性与长稳	6
	3.2 协议参数：定量推导（与指导书 11.3(3)(5) 对齐）	6
	3.3 性能测试记录（指导书表 3）	7
	3.4 实测与理论对比	7
4	11.4 研究和探索的问题（至少 2 题深度展开）	9
	4.1 探索一：CRC-32 漏检概率与工程意义（尹浩铭）	9
	4.2 探索二：start_timer 与 start_ack_timer 的设计差异（尹浩铭，结合 protocol.c）	9
5	11.5 实验总结和心得体会	11
	5.1 调试复盘（尹浩铭记录）	11
	5.2 分工（与《三天三人方案》一致）	11
	5.3 对实验库的建议	11
6	11.6 源程序文件	12
7	附录	12
	7.1 关键日志摘录	12

1 11.1 实验内容和实验环境描述

1.1 实验任务与目标

本实验在教师提供的物理层与网络层仿真库上，自行设计并实现数据链路层协议，使 A、B 两站在有误码的全双工信道上仍能向网络层交付无差错分组，并完成指导书要求的信道利用率测试与协议参数说明。

本组选定协议：搭载 ACK 的 **Go-Back-N (GBN)**。信道与分组参数与库一致（见 `include/protocol.h`、`src/protocol.c`）：

- 带宽 $B = 8000$ bps（全双工）；单向传播时延 $t_p = 270$ ms。
- 网络层分组长度 $N = 256$ 字节（`PKT_LEN`）；默认比特误码率 $P_b = 10^{-5}$ （可用命令行 `-b` 调整）。
- 数据帧在内存中为 `struct frame(kind、seq、ack、data[256])`，发送时在帧尾再追加 4 字节 CRC-32（与指导书 8.9 一致）。

复现命令示例（Linux 下于仓库根目录 `make` 后）：

```
# 终端 1 (A 站)
./datalink -d3 A
```

```
# 终端 2 (B 站)
./datalink -d3 B
```

无误码洪水、高误码等场景分别叠加 `--flood`、`--utopia`、`-b 1e-4` 等选项；正式报告中的命令行、运行时长须与你们留存日志一致。

1.2 实验环境

- 硬件与系统：（填写主机型号、内存、Windows 版本或 Linux 发行版）。
- 编译器：GCC（Makefile）或 Visual Studio（Lab1-Windows-VS2017 等工程）。
- 代码版本：（建议填写 `git rev-parse --short HEAD`），便于与附录日志、截图交叉核验。

与实验库的一致性说明：下文定时器、`phl_sq_len` 的叙述直接对应本仓库 `src/protocol.c`；滑动窗口状态机与宏常量以最终实现为准：`WINDOW_SIZE = 3`、`MAX_SEQ = 255`、`DATA_TIMEOUT_MS = 600`、`ACK_TIMEOUT_MS = 200`。

2 11.2 软件设计

2.1 （1）数据结构

字段 / 变量	类型	含义与设计要点
kind	unsigned char	区分 FRAME_DATA / FRAME_ACK 等；与组内枚举一致。
seq	unsigned char	发送序号；GBN 发送窗口内序号递增，模 (MAX_SEQ + 1)。
ack	unsigned char	累积确认语义须在组内冻结（常见为「期望下一帧序号」）。
data[PKT_LEN]	unsigned char	256 字节载荷；仅数据帧有效。
CRC 区	4 字节	CRC-32 (IEEE 802.3) 校验域：不在 struct frame 内重复定义，发送时由 crc32(buf, FRAME_HDR_LEN + PKT_LEN) 计算后追加到帧尾 4 字节；接收端以 recv_frame 读入长度为 FRAME_HDR_LEN + PKT_LEN + 4 的完整帧后，对整帧做 crc32(...) == 0 校验。详见 datalink_recv.c 中 validate_and_process_frame。
ack_expected	序号	发送窗口下界：最老未确认帧。
next_frame_to_send	序号	发送窗口上界：下一新帧序号（受窗口宽度约束）。
frame_expected	序号	接收端按序期望的下一 seq。
out_buf[]	帧缓存	GBN 重传：超时后从 ack_expected 起连续重发窗口内帧。

, caption: [帧与核心状态变量（与 include/datalink.h 及实现保持一致）],

序号空间与窗口：seq / ack 字段为 1 字节，本实现使用完整 $M = \text{MAX_SEQ} + 1 = 256$ 个编号（0–255），发送窗口仍取 $W = 3$ 。GBN 一般要求 $W \leq M - 1$ ，本实现远小于上限；较大的序号空间用于避免误码重传时旧帧在物理层队列中滞留、序号过早回绕后被接收端误认为新帧。

2.2 （2）模块结构

主控结构为：protocol_init → 主循环 wait_for_event → switch(event) 分发。关键接口来自 protocol.h：get_packet / put_packet、send_frame / recv_frame、定时器与 enable_network_layer / disable_network_layer。

模块 / 分支	主要职责	代码位置
send_one_data_frame	取网络层分组、组 DATA 帧、捎带 ACK、追加 CRC、缓存并发送	src/datalink.c:40
update_ack_received	按累积 ACK 推进发送窗口，维护数据定时器	src/datalink.c:22

resend_window	DATA_TIMEOUT 后从 ack_expected 起重传窗口内帧	src/datalink.c:73
FRAME_RECEIVED	调用 validate_and_process_frame, 再处理 ACK、按序上交、启动 ACK 定时器	src/datalink.c:123
ACK_TIMEOUT	调用 send_pure_ack 发送纯 ACK	src/datalink.c:148
refresh_network_layer_gate	根据窗口占用开关网络层	src/datalink.c:87

, caption: [事件循环与主要函数位置],

2.3 (3) 算法流程与异常路径

正常路径（示意）：NETWORK_LAYER_READY 时组数据帧、捎带 ack、附加 CRC、send_frame；对未确认序号 start_timer(seq, "DATA_TIMEOUT_MS")。FRAME_RECEIVED 且 CRC 正确时，用 ack 滑动发送窗口并 stop_timer；按序数据 put_packet，否则缓存策略按 GBN 丢弃或按序交付。

异常与边界（评审关注点）：

1. CRC 失败：recv_frame 返回后先做 `crc32(...)==0` 判断；失败则静默丢弃，不推进 frame_expected，不错误调用 put_packet（避免网络层坏分组）。可配合 dbg_event 记录丢帧。
2. 序号跳变（缺口）：若收到 `seq != frame_expected`，GBN 丢弃并重复发送累积 ACK（语义依组内定义），促使发送端回退到正确窗口边界。
3. 重复帧：`seq < frame_expected` 或已交付过的序号：不重复上交网络层，可发重复 ACK 以驱动对端窗口滑动。
4. DATA_TIMEOUT：从 ack_expected 起重传；每次有效重传后须重新 start_timer，否则若重传帧再次损坏，可能再无超时事件，形成「静默死锁」（见 11.5 复盘）。

流量控制：enable_network_layer / disable_network_layer

网络层是否产生 NETWORK_LAYER_READY，受 enable 标志与仿真库内业务模型共同影响；链路层侧应在发送窗口已满、待发队列过长或协议状态不允许取新分组时 disable_network_layer()，在窗口有空间且逻辑允许发送新帧时 enable_network_layer()。

与物理层队列 phl_sq_len() 的关系：本仓库 src/protocol.c 中，数据帧重传定时器的到期时刻为

$$t_{\text{expire}} = t_{\text{now}} + \text{phl_sq_len}() \cdot 8000 / \text{CHAN_BPS} + \text{ms}$$

其中第二项把当前物理层发送队列中尚未离站的字节数按信道速率折算为排队发送所需时间（与 src/protocol.c 中 start_timer 一致：CHAN_BPS 为 8000 时，系数为 1，量级上「每字节约 1 ms」发送时间）。这样 start_timer 在本地排队较长时会自动推迟超时点，减少虚假超时重传。验收报告应写清：你们在 datalink.c 中何时调用 start_timer / stop_timer，以及是否与上述库语义一致。

PHYSICAL_LAYER_READY 在库中当 `phl_sq_len() < "PHL_SQ_LEVEL"`（本仓库 PHL_SQ_LEVEL = 50 字节）等条件满足时通知链路层可继续向物理层写字节；链路层应利用该事件把积压帧写出，避免队列逼近 SQ_SIZE 上限时触发 ABORT("... overflow")。

3 11.3 实验结果分析

3.1 有误码下的正确性与长稳

1. 有误码时，网络层不应出现指导书 8.13 中的 *bad packet* 异常退出（若出现，须记录场景与日志并说明修复）。
2. 长稳：建议单场景连续运行 ≥ 10 分钟（报告可写 20 分钟量级），附录贴 `datalink-A.log` / `datalink-B.log` 首尾时间戳与末次 `put_packet` 统计。

3.2 协议参数：定量推导（与指导书 11.3(3)(5) 对齐）

设一帧在信道上的比特长度为 L_{bit} （含首部与 CRC）。工程上常取：首部 3 字节 + 载荷 256 字节 + CRC 4 字节 $\Rightarrow L_{\text{byte}} = 263$, $L_{\text{bit}} = 2104$ 。

（1）单帧发送时间

$$t_{\text{tx}} = \frac{L_{\text{bit}}}{B}$$

代入 $B = 8000$ bps，得 $t_{\text{tx}} = \frac{2104}{8000} \approx 0.263$ s（约 263 ms）。若报告中用「仅载荷 2048 bit」近似，则 $t_{\text{tx}} = 0.256$ s；须在表格脚注中声明采用了哪种口径，全文口径一致。

（2）带宽时延积（单向）

$$\text{BDP} = B \cdot t_p = 8000 \times 0.27 = 2160 \text{ bit}$$

约合 $\frac{2160}{L_{\text{bit}}} \approx 1.03$ 帧（在途「管道」可容纳的未完成传播比特量）。

（3）窗口下界（填满管道）

GBN 在持续发送时，为在首个 ACK 返回前不因窗口阻塞而空等，常采用（与指导书及教材等价的）必要条件：

$$W \cdot t_{\text{tx}} \geq 2t_p + t_{\text{tx}} \quad \text{即} \quad W \geq \frac{2t_p + t_{\text{tx}}}{t_{\text{tx}}}$$

代入 $t_p = 0.27$ s、 $t_{\text{tx}} \approx 0.263$ s，得 $W \geq \frac{0.54+0.263}{0.263} \approx 3.05$ ，故教材口径下取 $W = 4$ 更保守。本组现在 `include/datalink.h` 中取 `WINDOW_SIZE = 3`：按字节 BDP 估算，RTT 内比特量 $8000 \times 0.54 \approx 4320$ bit，约合 540 字节；若整帧约 260 字节则 $\frac{540}{260} \approx 2.08$ ，向上取 3，与代码一致；误码场景下较小窗口可略减 GBN 回退跨度（与「取 4 提高无误码利用率」的折中须在正文写清）。

（3.1）定时器宏与报告一致

数据帧重传超时取 `DATA_TIMEOUT_MS = 600`（略大于 $\text{RTT} \approx 540$ ms 加处理余量；库中 `start_timer` 另含 `phl_sq_len` 排队项，见 `src/protocol.c`）。搭载 ACK 等待时间取 `ACK_TIMEOUT_MS = 200`，避免大 RTT 下过久等待捎带而导致对端不必要的超时重传。

（4）停等与滑动窗口的理论利用率（简化模型）

- 停等 ($W = 1$): $\eta_1 \approx \frac{t_{\text{tx}}}{t_{\text{tx}} + \text{RTT}}$ ，若取 $\text{RTT} \approx 2t_p + t_{\text{tx}}$ （忽略短 ACK），则 $\eta_1 \approx \frac{t_{\text{tx}}}{2t_p + 2t_{\text{tx}}}$ 。代入 $t_p = 0.27$ 、 $t_{\text{tx}} = 0.263$ 得约 32% 量级（与「管道极空」直觉一致）。
- 窗口为 W 的流水线（无误码、ACK 不阻塞的理想化）：当 W 足够大时，上限趋近于 $\frac{t_{\text{tx}}}{t_{\text{tx}} + \frac{\text{RTT}}{W}}$ 等形式；一种便于制表的近似为

$$\eta_W \approx \min\left(1, \frac{W \cdot t_{tx}}{t_{tx} + \text{RTT}}\right)$$

或按指导书给定公式推导。建议林旭东做一张「 $W \in \{1, 2, 4, 8\} \rightarrow$ 理论 η 」小表，并与洪水场景实测利用率同图对比（Excel / Python 出图后插入 PDF）。

（5）有误码时的利用率上界与雪崩

设比特误码率为 P_b ，帧长 L_{bit} 较大时，一帧无错概率可近似

$$P_{\text{ok}} \approx (1 - P_b)^{L_{\text{bit}}} \approx e^{-L_{\text{bit}} P_b}$$

平均每成功一帧所需发送次数约 $\frac{1}{P_{\text{ok}}}$ 。在独立误码假设下，有效吞吐量约按 P_{ok} 比例缩放。

当 P_b 升高时，GBN 一旦发生 CRC 失败或丢 ACK，往往要从窗口下界重传多帧，重传帧再次出错的概率叠加，利用率随 P_b 上升呈陡降（俗称相对 SR 的「雪崩」效应）。报告须用一两段话把该因果链写清，并与表 3 中场景 4 vs 5 的落差对照。

3.3 性能测试记录（指导书表 3）

序号	场景	A 命令	B 命令	A 利用率 %	B 利用率 %	运行时长 / 备注
1	无误码	-u -d0 -t 600 -p 59281 -l table3-1-utopia-A.log A	-u -d0 -t 600 -p 59281 -l table3-1-utopia-B.log B	39.36	69.64	600 s; Err 0
2	默认业务	-d0 -t 600 -p 59282 -l table3-2-default-A.log A	-d0 -t 600 -p 59282 -l table3-2-default-B.log B	34.18	61.81	600 s; Err 20/33
3	双端洪水+无误码	-f -u -d0 -t 600 -p 59283 -l table3-3-flood-utopia-A.log A	-f -u -d0 -t 600 -p 59283 -l table3-3-flood-utopia-B.log B	72.32	72.30	600 s; Err 0
4	双端洪水（默认误码）	-f -d0 -t 600 -p 59284 -l table3-4-flood-default-A.log A	-f -d0 -t 600 -p 59284 -l table3-4-flood-default-B.log B	63.69	62.80	600 s; Err 32/34
5	洪水+高误码	-f -b 1e-4 -d0 -t 600 -p 59285 -l table3-5-flood-berle-4-A.log A	-f -b 1e-4 -d0 -t 600 -p 59285 -l table3-5-flood-berle-4-B.log B	32.02	31.24	600 s; Err 264/255

,caption: [表 3 实测数据：五场景均自然 Quit，未出现 bad packet / Abort。利用率取日志末次 packets received 行中 bps 与 8000 bps 的比例。],

结果分析。场景 3 在无误码且双端持续洪水时，两端利用率稳定在约 72.3%，说明数据链路层可持续推进窗口且没有死锁。场景 4 在默认误码下仍保持 63% 左右，CRC 错误被丢弃并通过 GBN 超时重传恢复。场景 5 将误码率升至 10^{-4} 后利用率降到约 31%–32%，符合 GBN 在高误码下「一处出错、窗口回退、多帧重传」导致吞吐下降的预期。

3.4 实测与理论对比

场景	理论参考 %	实测 %	差距	原因分析要点
3 洪水无误码	载 荷 上 界 $\frac{256}{263} \approx 97.34$	72.31	约 -25.0	WINDOW_SIZE=3、ACK 等待、事件调度与物理层排队使实现低于理想满载

场景	理论参考 %	实测 %	差距	原因分析要点
4 洪水默认误码	$7.34 \times e^{-2104 \times 10^{-5}}$ 95.31	≈ 63.25	约 -32.1	误码触发 CRC 丢帧与 GBN 回退, 实际代价大于独立成功率缩放
5 洪水 10^{-4}	$7.34 \times e^{-2104 \times 10^{-4}}$ 78.95	≈ 31.63	约 -47.3	高误码下窗口内多帧被重复发送, 重传放大明显

, caption: [理论值为简化上界, 仅用于解释数量级; 实测值取 A/B 末次利用率均值],

4 11.4 研究和探索的问题（至少 2 题深度展开）

4.1 探索一：CRC-32 漏检概率与工程意义（尹浩铭）

CRC-32（IEEE 802.3 标准，生成多项式 $G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ ）在实验中的使用方式为：发送端在帧末尾追加 4 字节 CRC 校验值，接收端对整帧（含 CRC 字段）计算 `crc32(...)`，若结果不为 0 则判定帧已损坏。

漏检概率的理论估计。CRC-32 的漏检概率 $\approx 2^{-32} \approx 2.33 \times 10^{-10}$ （假设错误模式均匀随机）。设链路利用率为 η ，信道速率 $B = 8000$ bps，帧长 $L_{\text{bit}} = 2104$ ，则每秒发送帧数约 $\eta \frac{B}{L_{\text{bit}}}$ 。取 $\eta = 0.5$ 得：

$$N_{\text{day}} \approx \frac{0.5 \times 8000 \times 86400}{2104} \approx 1.64 \times 10^5 \text{ 帧/天}$$

漏检一帧所需的平均天数约为：

$$T_{\text{漏检}} \approx \frac{1}{N_{\text{day}} \times 2^{-32}} \approx \frac{1}{1.64 \times 10^5 \times 2.33 \times 10^{-10}} \approx 2.62 \times 10^4 \text{ 天} \approx 72 \text{ 年}$$

这意味着在 $P_b = 10^{-5}$ 的随机误码模型下，CRC-32 漏检在统计上可忽略；实际运行中观察到的 bad packet 几乎全部来自 CRC 校验正确而实现的逻辑错误（如缓冲区长度不对、put_packet 调用条件错误）。

与 P_b 导致的 CRC 可检出错误率的对比。默认误码率 $P_b = 10^{-5}$ 时，一帧（2104 bit）至少出现 1 bit 错误的概率为：

$$P_{\text{err}} = 1 - (1 - P_b)^{L_{\text{bit}}} \approx 1 - e^{-L_{\text{bit}} P_b} \approx 1 - e^{-0.02104} \approx 0.0208 \approx 2.08\%$$

即约每 48 帧中就有 1 帧被 CRC 检测出错误并丢弃。相比之下，漏检概率 $\sim 2.33 \times 10^{-10}$ 比可检出错误概率低约 8 个数量级。因此 CRC-32 在本实验场景中足够可靠。

更高误码的退化。当 P_b 升至 10^{-4} 时， $P_{\text{err}} \approx 1 - e^{-0.2104} \approx 0.189$ ，约 18.9% 的帧被 CRC 标记为损坏。GBN 协议下 CRC 错误帧导致回退重传整窗口，利用率急剧下降（见表 3 场景 4→5 的落差），但这属于协议层面的「雪崩」效应，而非 CRC 校验本身的不足。

4.2 探索二：start_timer 与 start_ack_timer 的设计差异（尹浩铭，结合 protocol.c）

根据本仓库 src/protocol.c 中的实现，两种定时器的设计存在以下关键差异：

（1）超时基准不同。start_timer(nr, ms) 的超时时刻为：

$$t_{\text{expire}} = t_{\text{now}} + \text{phl_sq_len}() \times \frac{8000}{\text{CHAN_BPS}} + \text{ms}$$

其中 phl_sq_len() 返回当前物理层发送队列中尚未离站的字节数，按信道速率折算后再计参数 ms。这样做的目的是：如果本地物理层排队较长，自动推迟超时点，避免「帧还在队列中未来得及上线路就超时」导致的虚假重传。而 start_ack_timer(ms) 的到期时刻仅从当前时刻起算 ms，不含排队折算，因为 ACK 帧很短（纯 ACK 仅 7 字节），在等待 ACK_TIMEOUT 的 200 ms 内通常已完成发送。

(2) 触发和重置策略不同。`start_timer` 在发送一帧或收到有效 ACK 后无条件更新定时器 (见 `datalink.c` 中 `send_one_data_frame` 和 `update_ack_received`)。但 `start_ack_timer` 的实现 (`protocol.c` 第 560 行) 为:

```
if (timer[ACK_TIMER_ID] == 0)
    timer[ACK_TIMER_ID] = now + ms;
```

即仅当 ACK 定时器未运行时才启动, 对已运行的定时器不重置。这样在连续收到数据帧的场景下, ACK 定时器只启动一次, 不会因每次 `FRAME_RECEIVED` 都重置而无限推迟。当捎带 ACK 随下一个数据帧返回对端后, 停止 ACK 定时器即可。

(3) 设计意图的工程意义。

定时器类型	触发条件	核心语义	防止的问题
<code>start_timer</code>	发出数据帧后	排队延迟后计时, 确保真实超时	虚假重传
<code>start_ack_timer</code>	CRC 正确且收到有效 DATA 帧后	一次启动, 不频繁重置, 确保最终发送纯 ACK	捎带 ACK 无限拖延

(4) 错误使用场景推演。若在 `ACK_TIMEOUT` 分支中错误调用 `start_timer` 代替 `start_ack_timer` 来管理纯 ACK 发送, `phl_sq_len` 的排队项将导致 ACK 超时被人为推迟; 在全双工洪水业务下, 发送队列可能持续非空, 纯 ACK 超时可能被大幅延迟, 甚至造成对端发送窗口停滞。若对 `start_ack_timer` 频繁手动重置 (即去掉 `if (timer[ACK_TIMER_ID] == 0)` 的条件), 则在洪水场景下 ACK 定时器可能被不断推后, 纯 ACK 始终不发送, 窗口滑动完全依赖反向数据帧的捎带, 一旦反向数据暂停, 确认信息将无法回传。

(5) 在本实现中的体现。本组代码 `datalink_recv.c` 实现了 `validate_and_process_frame`, 当收到有效 DATA 帧 (`rc == 1` 或 `rc == 2`) 时调用 `start_ack_timer(ACK_TIMEOUT_MS)`, 由库保证不重置已运行的定时器。`datalink.c` 的 `ACK_TIMEOUT` 分支调用 `send_pure_ack` 发送纯 ACK 后 `stop_ack_timer`, 二者分工清晰。

5 11.5 实验总结和心得体会

5.1 调试复盘（尹浩铭记录）

CRC 校验与帧长问题。首版编译后运行出现段错误，定位为 `recv_frame` 的缓冲区不足：接收缓冲区仅 256 字节，但含 CRC 的完整 DATA 帧为 3（首部）+ 256（载荷）+ 4（CRC）= 263 字节。将 `rxbuf` 扩大为 512 字节后解决。后续又将帧结构中的魔术数 3 替换为宏 `FRAME_HDR_LEN`，避免在全文件范围硬编码。

纯 ACK CRC 遗漏。调试初期观察到对端偶发收到纯 ACK 帧后 CRC 校验失败：原因是 `send_pure_ack` 中仅填充了 3 字节的首部就直接调用了 `send_frame`，未附加 CRC。修复后在帧尾计算并追加 4 字节 CRC。

CRC 校验与 `recv_frame` 长度的关系。一个较隐蔽的问题是：`recv_frame` 返回的长度参数需要包含末尾 CRC 的 4 字节，才能对整帧做 `crc32(...) == 0` 校验。早期版本在 `validate_and_process_frame` 中未拉长 `recv_frame` 的 `size` 参数，导致 `len` 不包含 CRC 域，CRC 校验始终失败。修正后接收路径恢复正常。

失序帧处理。GBN 协议要求接收端仅接收按序到达的帧，失序/重复帧应丢弃并重复发送 ACK。初始实现中失序帧虽然未调用 `put_packet`，但错误地推进了 `frame_expected`，导致序号持续偏移、`put_packet` 全部失败。修正后在 `validate_and_process_frame` 中增加 `return 2` 分支：失序帧保持原 `frame_expected` 不变，调用方（`datalink.c` 的 `FRAME_RECEIVED` 分支）仍会处理捎带的 `ack_seq` 并启动 `ACK_TIMEOUT`，确保对端能收到重复的确认。

序号空间过小导致旧帧误收。一次默认误码长跑中，早期 `MAX_SEQ=7` 的版本在约 40 秒出现 `Network Layer received a bad packet`。复查日志后判断：GBN 超时重传会把窗口内帧重新排入物理层队列，若旧重传帧滞留到接收端序号快速绕回，就可能被误认为下一轮的新帧并错误上交网络层。最终使用 `unsigned char` 的完整 0-255 序号空间，将 `MAX_SEQ / NR_BUFS` 调整为 255 / 256，窗口大小仍为 3。复测表 3 五场景各 600 秒均自然退出，无坏分组。

5.2 分工（与《三天三人方案》一致）

- 张恒基：事件循环、GBN 发送窗口与超时/重传逻辑；保证流程图与代码行号一致。
- 尹浩铭：帧格式（`include/datalink.h`）、`FRAME_HDR_LEN` 宏、CRC 集成；`datalink_recv.c` 全模块（CRC 校验、重复/失序检测、按序交付 `put_packet`、纯 ACK 组帧与携带 ACK 维护）；排查结构体布局与缓冲区内存问题；撰写 11.4 探索一（CRC 漏检）、探索二（定时器设计差异）及调试复盘。
- 林旭东：表 3 五场景数据、长稳日志、曲线与 `Makefile`/运行矩阵。

5.3 对实验库的建议

（简述希望改进的日志接口、统计项等，1-2 段即可。）

6 11.6 源程序文件

列出提交的自写文件：src/datalink.c、include/datalink.h 等；教师库 src/protocol.c、include/protocol.h 勿算作自研行数但须在报告中引用其接口语义。

关键宏与代码一致性检查清单（实现完成后逐项打勾）：

窗口宏 WINDOW_SIZE / MAX_SEQ 与报告 11.3 推导一致	✓
DATA_TIMEOUT_MS、ACK_TIMEOUT_MS 与推导一致并在注释中写明 RTT 估算	✓
表 3 命令行、利用率与日志文件名一致	✓
流程图标注的 datalink.c 行号已更新	✓
附录含 600 秒稳定运行摘录	✓
研究与探索 ≥ 2 题，至少一题含量化或小程序	✓

7 附录

7.1 关键日志摘录

以下为表 3 末次统计行摘录，日志均在仓库根目录运行时生成：

```
table3-1-utopia-A.log      598.908 .... 919 packets received, 3149 bps, 39.36%, Err 0
(0.0e+00)
table3-1-utopia-B.log      598.924 .... 1628 packets received, 5571 bps, 69.64%, Err
0 (0.0e+00)
table3-2-default-A.log     599.042 .... 798 packets received, 2734 bps, 34.18%, Err
20 (1.0e-05)
table3-2-default-B.log     599.073 .... 1444 packets received, 4945 bps, 61.81%, Err
33 (1.0e-05)
table3-3-flood-utopia-A.log 599.383 .... 1691 packets received, 5786 bps, 72.32%, Err
0 (0.0e+00)
table3-3-flood-utopia-B.log 599.570 .... 1691 packets received, 5784 bps, 72.30%, Err
0 (0.0e+00)
table3-4-flood-default-A.log 598.565 .... 1488 packets received, 5095 bps, 63.69%, Err
32 (9.4e-06)
table3-4-flood-default-B.log 599.669 .... 1470 packets received, 5024 bps, 62.80%, Err
34 (1.0e-05)
table3-5-flood-berle-4-A.log 598.441 .... 748 packets received, 2561 bps, 32.02%, Err
264 (1.0e-04)
table3-5-flood-berle-4-B.log 595.204 .... 726 packets received, 2500 bps, 31.24%, Err
255 (9.9e-05)
```